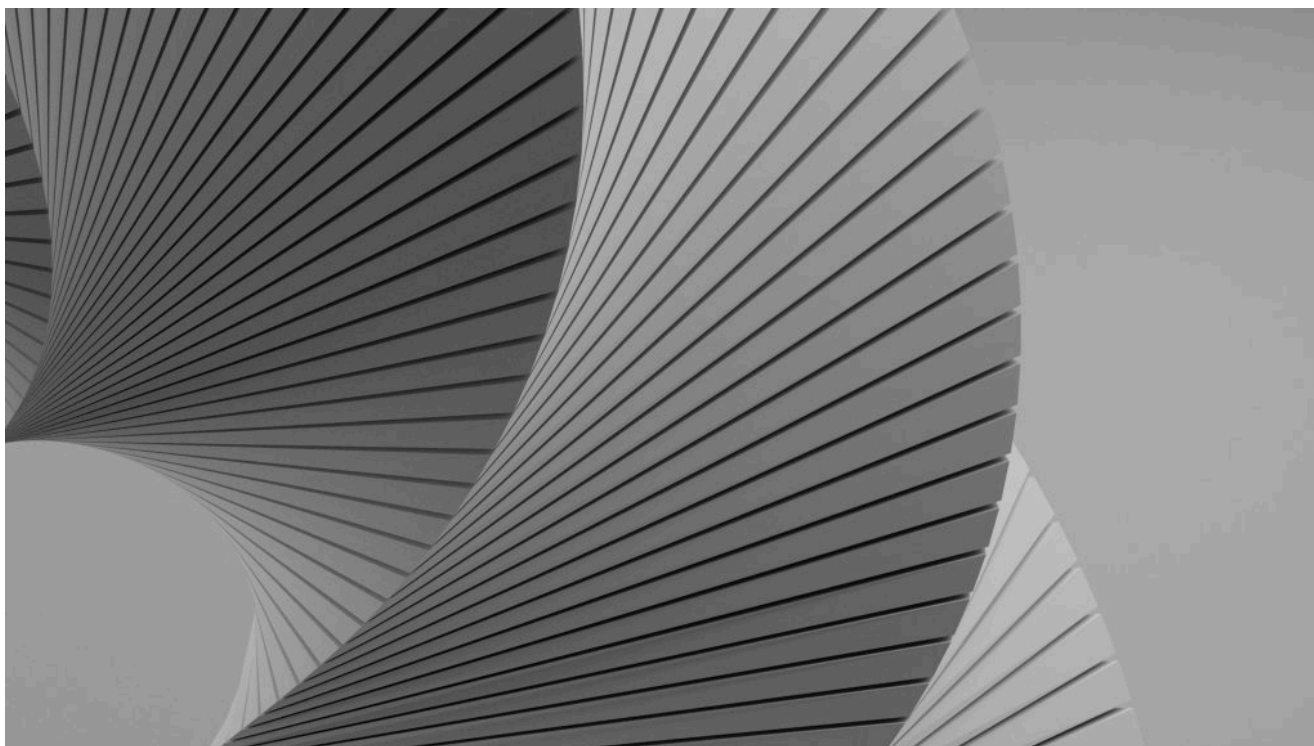


РАЗРАБОТКА ИИ / ИНФРАСТРУКТУРА ИИ / БЭКЕНД-РАЗРАБОТКА

Персонализация — это проблема ранжирования, и архитектура помогает ее решить

Персонализация не работает из-за архитектуры, а не из-за некачественных данных. Узнайте, как ранжирование в реальном времени обеспечивает мгновенную релевантность с учетом контекста.

23 июля 2026 года, 12:00 [Дженни Моррис](#)



Филип Орони для Unsplash+

[Vespa.ai](#) спонсировала этот пост.

Каждая продуктовая команда стремится к одному и тому же моменту: пользователь открывает страницу и думает: «Это понимает меня».

Покупатель, которому нравятся цветочные принты, должен видеть больше цветочных принтов. Пользователь, интересующийся местной политикой, должен открывать свое приложение, чтобы читать новости о местной

Это уже не нишевая функция. Это базовое требование. Пользователи быстро определяют, понимает ли их система продукта, и их редко волнует, в чем причина сбоя — в поиске, рекомендациях, правилах мерчандайзинга или устаревших данных.

Вот неприятная правда: у большинства команд нет проблемы с персонализацией качества. У них есть проблема с персонализацией архитектуры.

Персонализация — это не виджет, прикрепленный к поиску. Это решение о ранжировании. Система должна решить для конкретного пользователя и конкретного запроса, что заслуживает следующего места в выдаче. Это означает, что система одновременно учитывает данные о пользователе, объекте, контексте и бизнес-цели. Во многих стеках уровень ранжирования — это единственное место, где все эти сигналы не учитываются одновременно.



Vespa.ai — это платформа для создания приложений на основе искусственного интеллекта для поиска, рекомендаций, персонализации и управления клиентским опытом. Она обрабатывает большие объемы данных и выполняет множество запросов, обеспечивая эффективное управление данными, логическими выводами и логикой. Доступна как в виде управляемого сервиса, так и с открытым исходным кодом.

[Узнать больше →](#)

ПОСЛЕДНИЕ НОВОСТИ ОТ VESPA.AI

[Ваш агент хочет искать работу, как квант в 2010 году](#)

7 июля 2026 года

[Повторное автоматическое исследование MSMARCO BM25 на Vespa](#)

29 мая 2026 года

[Информационный бюллетень Vespa, май 2026 года](#)

27 мая 2026 года

[Отправить](#)

Самое сложное — не сбор сигналов. Самое сложное — их объединение, пока пользователь находится в сети.

Почему персонализация — это сложно

Чтобы поместить нужный товар в нужное место, система должна понимать сразу несколько вещей:

- **Цель:** Что пользователь запрашивает прямо сейчас?
- **Качество товара:** Что на самом деле содержит или представляет каждый кандидат?
- **История пользователя:** Что этот человек нажимал, покупал, читал, смотрел или игнорировал?
- **Доступность:** Есть ли товар в наличии, свежий ли он, находится ли он поблизости, можно ли его показывать и готов ли он к отправке?
- **Приоритеты бизнеса:** Что бизнес должен продвигать, защищать или, наоборот, не акцентировать на этом внимание?

Эти сигналы часто противоречат друг другу. Самый релевантный товар может оказаться не самым прибыльным. Самый прибыльный товар может быть вне ассортимента. Пользователь может написать «кроссовки для бега», но его поведение говорит о том, что ему нужны «кроссовки для трейлраннинга, широкий крой, до 120 долларов».

ТРЕНДОВЫЕ ИСТОРИИ

1. **Код доставки без проверки человеком**
2. **«Пора избавиться от человеческого мусора»: почему ИИ теперь проверяет код лучше, чем ваш коллега.**

4. Почему «скучная» база данных — это ваша секретная суперсила в области ИИ

5. Как отказаться от проверки кода

Кроме того, они живут по разным часам. Атрибуты продукта меняются медленно. Запасы и цены могут меняться в течение дня. Предпочтения меняются с каждым кликом. Внешний контекст — погода, экстренные новости, матч чемпионата, культурный феномен — может повлиять на ситуацию без предупреждения.

Персонализация подразумевает объединение всего этого в один упорядоченный список при каждом запросе за миллисекунды. Сами по себе сигналы не являются узким местом. Узким местом является ранжирование во время запроса.

Обычный стек усложняет задачу

Большинство систем персонализации состоят из инструментов, каждый из которых предназначен для одного аспекта релевантности.

Поисковые системы по ключевым словам отлично справляются с лексическим сопоставлением. Они хороши, когда язык запроса совпадает с языком каталога. Но покупатели, читатели и соискатели редко используют точные термины, которые можно найти в индексе. Вы проиндексировали «спортивную беговую обувь», а они ввели «кроссовки». Правила использования синонимов могут помочь, но они не масштабируются должным образом при использовании длинных запросов, изменении каталогов и новых моделях поведения пользователей.

Векторные базы данных работают с противоположной стороны. Они хорошо справляются с семантическим сходством: «Найдите мне что-то похожее». Это мощный инструмент, но поиск ближайших соседей — это не то же самое, что персонализация. Настоящий ранжирующий алгоритм должен сочетать семантическое сходство с динамикой поведения, акциями, ценой, маржинальностью, актуальностью, соответствием требованиям и бизнес-правилами.

здесь и возникает фрагментация.

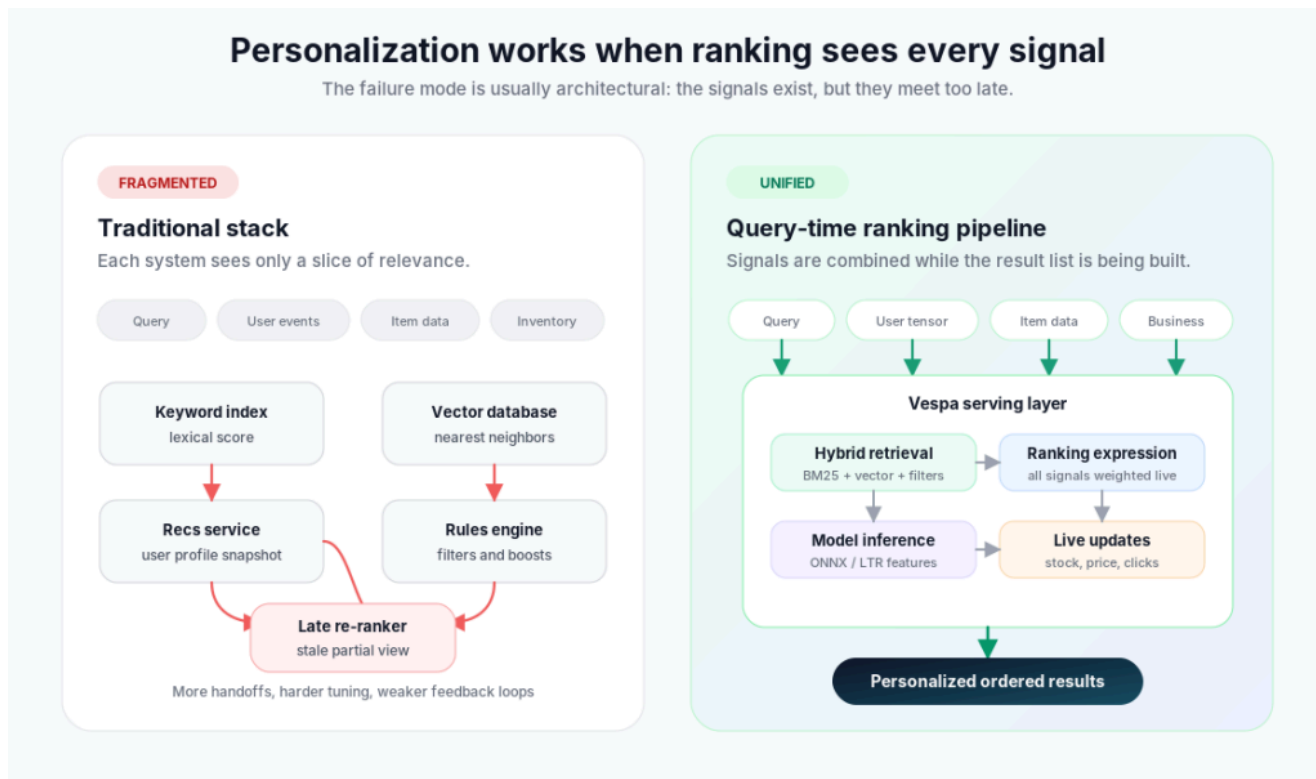


Рисунок 1. Фрагментированный стек персонализации в сравнении с унифицированным конвейером ранжирования во время запроса

Когда поиск и ранжирование осуществляются в разных системах, ранжировщик часто работает с неполной, устаревшей или предварительно вычисленной информацией. История кликов, контекст сеанса и вектор текущих предпочтений пользователя поступают слишком поздно. Бизнес-правила становятся фильтрами или переопределениями, а не сигналами для ранжирования. Обновление ассортимента или изменение цен требуют координации между несколькими системами.

Каждая передача данных увеличивает задержку. Каждая граница создает еще одно место, где сигналы могут исказиться. Каждое «быстрое правило» становится еще одним жестким ограничением, которое может случайно превратить «показать наиболее близкое совпадение» в «не показывать ничего».

«Каждая передача данных увеличивает задержку. Каждая граница создает еще одно место, где сигнал может

Более серьезная проблема связана с допущением о времени. Многие архитектуры строились на офлайн-ранжировании: обработка каталога, вычисление оценок в пакетном режиме и использование этих оценок до следующего перестроения. Это работает, когда предпочтения стабильны. Но система дает сбой, когда самый ценный сигнал — это клик, совершенный две секунды назад.

What changes when ranking happens in one real-time pipeline

A real-time personalization architecture treats retrieval, ranking, and inference as one serving problem.

That is the core idea behind **Vespa's approach**: Text search, vector similarity, structured filtering, ranking expressions, tensor computation, and model inference can live inside one query pipeline. Instead of retrieving somewhere, enriching somewhere else, and ranking at the end, the system can rank with the relevant signals while it is still deciding what to return.

That architectural choice changes the shape of the problem.

1. Retrieval is hybrid from the start

Lexical search, semantic search, and structured filtering can run together instead of being reconciled after the fact. A product query can combine text, embeddings, filters, session behavior, and item attributes in one request.

That matters because personalization is rarely one signal. The user's query still matters. So does semantic similarity. So do category, availability, price, and business constraints. Hybrid retrieval keeps those signals in play before ranking starts.

2. Ranking can express the actual objective

A personalization score should not be trapped inside one **similarity function**. It should be a formula that reflects the product's goals.

That formula might combine BM25, vector similarity, user affinity, stock level, margin, popularity, discount depth, freshness, rating, distance, or a weather term.

The important part is that they are all terms in the same ranking expression, not scattered across services.

A simplified version might look like this:

```
final_score =  
    0.30 * lexical_relevance +  
    0.25 * semantic_similarity +  
    0.25 * user_affinity +  
    0.10 * availability +  
    0.10 * business_priority
```

In production, the formula can be more nuanced. But the principle is simple: personalization, relevance, and business logic belong in the same scoring decision.

3. Model inference can run where the data lives

Some signals should come from learned models rather than hand-tuned rules: propensity to buy, churn risk, quality prediction, fraud risk, query classification, or a learned-to-rank model.

When inference runs in the serving path, those model outputs can become ranking features instead of delayed batch scores. That reduces the need to ship data to a separate inference service, wait for a response, and stitch the score back into ranking.

4. Updates become immediately useful

“Real time” should not mean “after the next index rebuild.” If inventory changes, stock should be rankable immediately. If a user clicks two yellow dresses, “yellow” should matter on the next request. If a merchandising team adjusts a ranking weight if the weight is exposed as a query-time input, the experiment should start producing useful feedback right away.

That is the difference between personalization as a nightly job and personalization as a live ranking decision.

Tensors make the personalization concrete

most useful mental model is simple: represent the user and the item in the same feature space, then rank by how well they match.

means the same framework can represent semantic embeddings, product attributes, user preferences, business objectives, and model features.

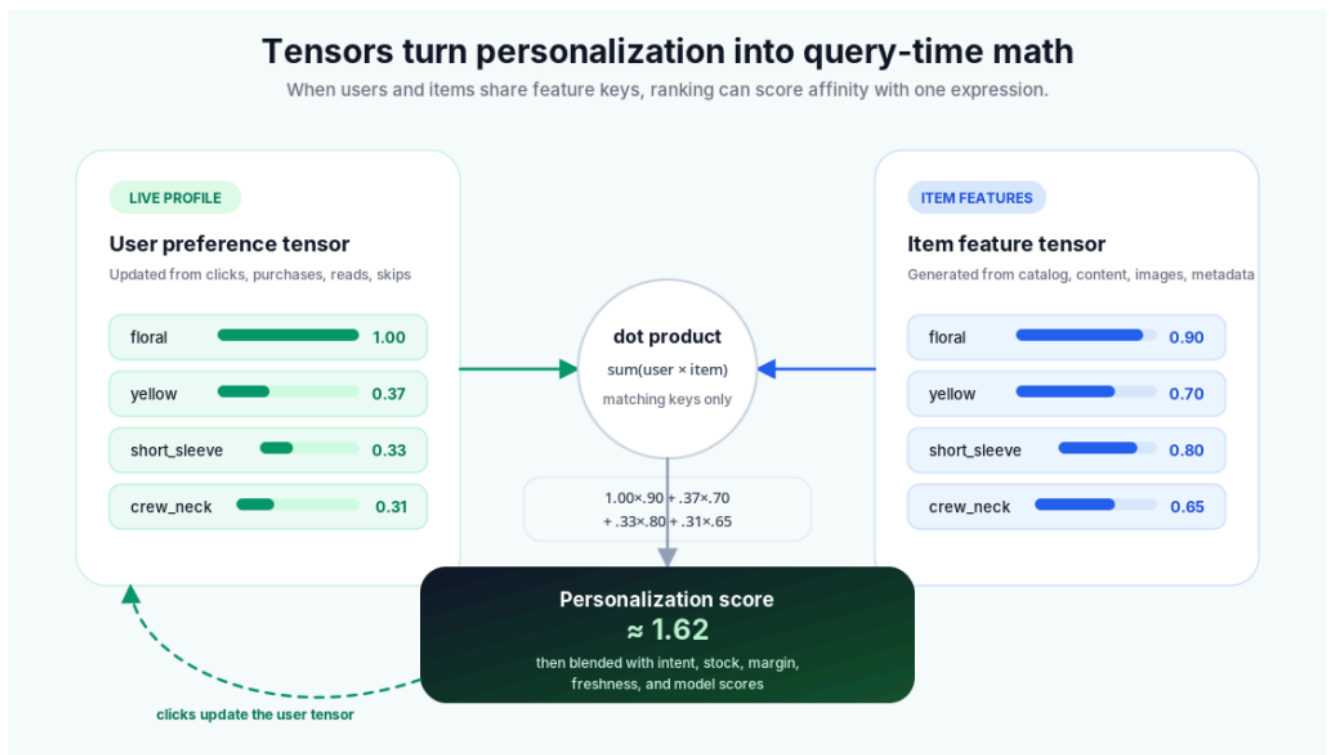


Figure 2. User and item tensors combined into a personalization score, then blended with other ranking signals

For example, each item can carry a sparse feature tensor:

```
{
  "floral": 0.90,
  "yellow": 0.70,
  "short_sleeve": 0.80,
  "crew_neck": 0.65
}
```

Each user can carry a tensor with the same feature names:

```
{
  "floral": 1.00,
  "yellow": 0.37,
  "short_sleeve": 0.33,
  "crew_neck": 0.31
}
```

Because the two tensors share a shape, personalization becomes a dot product: multiply matching features, sum the result, and use that score inside ranking.


```
# schema: item attributes stored as a sparse tensor
field item_features type tensor<float>(feature{}) {
  indexing: attribute | summary
}

# rank profile: the user's live preferences arrive as a query t
rank-profile personalized {
  inputs {
    query(user_features) tensor<float>(feature{})
  }
  first-phase {
    expression: sum(query(user_features) * attribute(item_f
  }
}
```

That one expression is not the whole ranking function. It is the personalization term. BM25, vector similarity, stock, margin, freshness, distance, or a model score can be added as other terms with their own weights.

The user tensor is where real-time behavior becomes powerful. Click a floral item, and the “floral” weight rises. Click two yellow items, and “yellow” rises; the application feeds click events into the user profile. The next query can use those updated preferences immediately, without waiting for a nightly profile build.

Business goals stop fighting personalization

In fragmented stacks, business rules often become blunt instruments: boost this category, hide that brand, force these items to the top, filter these out. That can satisfy a short-term merchandising goal while damaging relevance.

When business logic is part of the ranking expression, it can be more subtle. You can boost overstocked inventory without ignoring intent. Promote umbrellas when rain is forecast without turning every search into an umbrella search. Give new sellers a small exploration boost. Prioritize destocking before a new product line launches. Surface team merchandise during a championship run.

“When business logic is part of the ranking expression, the user still gets relevant results. The business still influences outcomes.”

systems.

That also makes experimentation easier. A merchandising or growth team can test weights, traffic splits, and ranking profiles without asking engineering to rewrite the whole pipeline. Relevance becomes a controllable growth lever rather than a fragile side effect.

The same pattern applies beyond commerce

The examples above are easy to picture in apparel, but the architecture is not commerce-specific. Personalization is the same ranking problem in many products:

- **Content feeds:** Blend topic affinity, freshness, engagement, creator quality, and business rules.
- **News:** Rank by reading history, topic interest, locality, freshness, and source diversity.
- **Jobs:** Match candidate preferences such as remote work, seniority, compensation, location, and tech stack against role attributes.
- **Geo search:** Treat distance as one normalized ranking term alongside relevance, quality, and preference.
- **Video and audio:** Combine embeddings, viewing history, metadata, freshness, and learned ranking models.

Different domains need different features. The architecture pattern is the same: retrieve candidates, rank with the signals that matter, update those signals as behavior changes, and keep the decision close to the data.

Scale doesn't have to be the trade-off

The natural concern is that a more expressive ranking system must be slower. In practice, that does not have to be true.

Vespa was built for large-scale serving from the beginning: billions of documents, high query volume, and low-latency ranking. The reason this works is multi-stage ranking. The system does not run the most expensive logic across every possible result. Instead, it uses a fast first phase to narrow the candidate set, then applies more precise ranking to the smaller group that remains.

operations, business logic, and model inference where they matter most.

The result is a practical balance: speed across the full corpus, accuracy in the final ranking, and enough flexibility to personalize each query without turning the serving stack into a chain of fragile services.

What's next

Personalization is not failing because teams lack data. Most teams already have plenty of signals: query intent, clicks, product attributes, inventory, margin, freshness, location, and business priorities. The harder problem is that those signals often live in different systems, move at different speeds, and arrive too late to influence the final ranking decision.

That is why personalization should be treated as a ranking problem. When retrieval, ranking, personalization, and business logic are split across separate systems, the ranker is forced to work with **stale or incomplete context**. The user moves faster than the architecture can respond. Every new signal becomes another integration project.

A unified real-time ranking pipeline changes that. User behavior, item attributes, semantic similarity, lexical relevance, inventory, and business goals can all become parts of the same scoring function. Tensors make those signals directly comparable and usable at query time. Instead of bolting personalization onto the end of the system, personalization becomes part of the decision the engine makes for every query.

The goal is simple: rank each result with the best context available, at the moment the user asks. That is when personalization stops feeling like a feature and starts feeling like relevance.



build scalable AI search, personalization, and retrieval-augmented generation (RAG) systems. With deep expertise in...

[Read more from Jenny Morris →](#)

TNS owner Insight Partners is an investor in: Real.

SHARE THIS STORY



TRENDING STORIES

1. Shipping code without human verification
2. "Time to clean up human slop": Why AI now reviews code better than your teammate.
3. The 800 mistakes that could reshape Meta's AI coding strategy
4. Why a 'Boring' Database Is Your Secret AI Superpower
5. How to kill the code review

INSIGHTS FROM OUR SPONSOR



Vespa.ai is a platform for building AI-driven applications for search, recommendation, personalization, and RAG. It handles large data volumes and high query rates, offering efficient data, inference, and logic management. Available as both a managed service and open source.

[Learn More →](#)

[AI agent wants to search like a 2010 quant](#)

July 2026

Vespa Newsletter, May 2026

27 May 2026

Scaling a Vespa Application: Feeding Fast and Furiously

28 April 2026

The Vespa Cloud Metrics Dashboard

24 April 2026

Using Large ONNX Models with External Data in Vespa Embedders

27 March 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the
most recent TNS articles in
your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

FOLLOW TNS



ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)